



Ejercicios de calentamiento lenguaje GO:

1. Ejercicio: El Guardián del Saldo (if / else)

Este ejercicio prepara al alumno para la Parte 1 (Validación), donde debe verificar condiciones de entrada

.

Objetivo: Determinar si una cuenta puede realizar un retiro.

```
package main
```

```
import "fmt"
```

```
func puedeRetirar(saldo float64, monto float64) {
```

```
    // Uso de if/else para lógica binaria simple
```

```
    if monto <= 0 {
```

```
        fmt.Println("Error: El monto debe ser positivo")
```

```
    } else if monto > saldo {
```

```
        fmt.Println("Error: Saldo insuficiente")
```

```
    } else {
```

```
        fmt.Printf("Retiro exitoso de %.2f. Saldo restante: %.2f\n", monto, saldo-  
monto)
```

```
    }
```

```
}
```

```
func main() {
```

```
    puedeRetirar(100.0, 50.0) // Caso exitoso
```

```
    puedeRetirar(100.0, 150.0) // Caso error saldo
```

```
}
```



2. Ejercicio: Clasificador de Monedas (switch)

Este ejercicio es la base para la Parte 2 (Cálculo de Comisión), donde se manejan múltiples métodos de pago

.

Objetivo: Mostrar el nombre completo de una moneda basándose en su código.

```
func obtenerNombreMoneda(codigo string) string {  
    // El switch en Go no necesita 'break' explícito  
    switch codigo {  
        case "PEN":  
            return "Soles Peruanos"  
        case "USD":  
            return "Dólares Americanos"  
        case "EUR":  
            return "Euros"  
        default:  
            return "Moneda desconocida"  
    }  
}
```

3. Ejercicio: Sumador de Transacciones (for range)

Prepara al alumno para la Parte 4 (Procesamiento de Lote), donde deben iterar sobre un arreglo de pagos

.

Objetivo: Calcular la suma total de un arreglo de números.

```
func sumarTransacciones(montos []float64) float64 {  
    total := 0.0
```



// for range devuelve el índice (i) y el valor (v)

```
// Usamos '_' para ignorar el índice que no necesitamos
for _, monto := range montos {
    total += monto
}
return total
}
```

4. Ejercicio: Limpieza de Datos (continue y break)

Este ejercicio es esencial para entender la Parte 5 (Control de Flujo), donde se deben filtrar montos inválidos o detener el proceso por seguridad

.

Objetivo: Procesar una lista de transacciones, ignorando las de 0 y deteniéndose si hay un error crítico (número negativo).

```
func procesarConSeguridad(datos []int) {
    for _, valor := range datos {
        if valor == 0 {
            fmt.Println("Saltando valor cero...")
            continue // Salta a la siguiente iteración
        }
        if valor < 0 {
            fmt.Println("¡ERROR CRÍTICO! Valor negativo detectado. Abortando.")
            break // Sale del bucle completamente
        }
        fmt.Printf("Procesando: %d\n", valor)
    }
}
```



}

```
func main() {  
    lista := []int{10, 0, 25, -1, 50}  
    procesarConSeguridad(lista)  
}
```

5. Ejercicio: Validador de Acceso (Condiciones Compuestas)

Ayuda a entender la Parte 3 (Clasificación de Riesgo), donde se evalúan varios criterios como el monto y el país simultáneamente

.

Objetivo: Verificar si un usuario tiene acceso VIP basándose en su antigüedad y sus compras.

```
func esVip(comprasTotales float64, añosAntigüedad int) bool {  
    // Uso de operadores lógicos && (AND) y || (OR)  
    if comprasTotales > 10000 || (comprasTotales > 5000 && añosAntigüedad > 2) {  
        return true  
    }  
    return false  
}
```